

Beyond Atomic Learning

Recommandations pour une plateforme francophone d'AI Engineering en 2026

1. Conclusion générale

Le modèle de freeCodeCamp est excellent pour apprendre :

- une syntaxe ;
- une API déterministe ;
- une opération précise ;
- une compétence dont le résultat peut être validé par un test binaire.

Il impose notamment un concept par challenge, un objectif réalisable en environ deux minutes, des tests immédiats et une progression cumulative. Ses workshops réutilisent le résultat de l'étape précédente, tandis que ses labs réduisent ensuite le niveau d'assistance.

Mais ce modèle ne doit pas être copié tel quel pour l'AI Engineering.

Un système agentique n'est pas seulement du code qui compile. C'est un système :

- probabiliste ;
- non déterministe ;
- dépendant d'un modèle et de son contexte ;
- capable d'emprunter plusieurs trajectoires valides ;
- soumis à des coûts, des limites, des erreurs d'outils et des risques ;
- dont la qualité ne peut pas être évaluée par un unique test `pass/fail`.

La plateforme ne devrait donc pas être un **freeCodeCamp consacré à l'IA**, mais un **simulateur d'ingénierie de systèmes agentiques**.

2. Ce qu'il faut conserver de freeCodeCamp

Quatre principes restent fondamentaux :

1. **Un objectif pédagogique clair à la fois.**
2. **Une action réelle immédiatement après l'explication.**
3. **Un feedback rapide et exploitable.**
4. **Une accumulation progressive vers un projet concret.**

Il ne faut pas supprimer l'Atomic Learning. Il faut l'intégrer dans une architecture pédagogique plus large.

3. Remplacer “Atom → Project” par “Atom → Loop → Graph → Mission → Operation”

C'est l'amélioration principale.

Niveau 1 — Atom

L'apprenant découvre une primitive isolée :

- définir un outil ;
- structurer une sortie ;
- créer une condition d'arrêt ;
- gérer un état ;
- ajouter un retry ;
- limiter un budget de tokens ;
- écrire un grader ;
- créer un handoff.

Durée indicative : 3 à 8 minutes.

Niveau 2 — Loop

L'apprenant combine plusieurs primitives dans une boucle complète :

Goal → Act → Observe → Verify → Continue or Stop

Exemples :

- boucle de recherche ;
- boucle de correction de code ;
- boucle génération-critique-révision ;
- boucle avec validation humaine ;
- boucle avec budget et terminal states.

Niveau 3 — Graph

L'apprenant conçoit plusieurs boucles connectées :

- routage ;
- branches conditionnelles ;
- fan-out/fan-in ;
- agents spécialisés ;
- état partagé ;
- délégation ;
- vérification indépendante ;
- escalade humaine.

Niveau 4 — Mission

L'apprenant reçoit un objectif métier et non une liste d'instructions techniques.

Exemple :

Construire un agent capable d'analyser un repository, de proposer une correction, de la tester et de produire une pull request vérifiable.

La plateforme fournit les contraintes, mais pas l'architecture.

Niveau 5 — Operation

L'apprenant doit faire fonctionner son système dans le temps :

- observer les traces ;
- détecter les régressions ;
- suivre les coûts ;
- corriger les échecs ;
- modifier les evals ;
- gérer une nouvelle version de modèle ;
- produire un postmortem.

Le projet ne s'arrête donc pas lorsqu'il fonctionne une fois.

4. Ne plus évaluer une réponse : évaluer un système

freeCodeCamp peut généralement vérifier qu'une fonction renvoie le bon résultat.

Pour un agent, il faut évaluer plusieurs dimensions :

```
evaluation:
  outcome_quality: 0-100
  task_success_rate: percentage
  tool_selection_quality: 0-100
  grounding_quality: 0-100
  safety_compliance: pass_fail
  recovery_from_failure: 0-100
  average_cost: currency
  token_usage: integer
  latency: milliseconds
  human_intervention_rate: percentage
```

Une mission ne devrait pas être exécutée une seule fois, mais sur un dataset de scénarios et, lorsque le comportement est stochastique, sur plusieurs runs.

Les systèmes d'évaluation modernes combinent justement des graders déterministes, des graders basés sur un modèle et du jugement humain. L'évaluation des traces permet également d'analyser les appels d'outils, les handoffs, les guardrails et les décisions internes, pas seulement la réponse finale.

La validation devrait donc fonctionner ainsi :

```
Code tests
+ outcome evals
+ trace evals
+ adversarial scenarios
+ cost constraints
+ occasional human review
```

5. Enseigner par les pannes, pas seulement par la construction

L'Atomic Learning classique montre généralement comment ajouter quelque chose correctement.

Pour former de véritables AI Engineers, au moins la moitié des exercices avancés devraient commencer avec un système défectueux.

Exemples :

- l'agent tourne indéfiniment ;
- il choisit le mauvais outil ;
- il perd son état ;
- il répète une action coûteuse ;
- il transmet trop de contexte ;
- son reviewer valide systématiquement le travail ;
- une injection de prompt détourne son objectif ;
- deux agents modifient simultanément le même fichier ;
- la boucle annonce `DONE` sans preuve ;
- le système fonctionne sur le cas de démonstration mais échoue sur les evals.

L'apprenant doit alors :

1. inspecter la trace ;
2. identifier la cause ;
3. formuler une hypothèse ;
4. modifier le système ;
5. prouver que la correction évite la régression.

Les traces sont devenues un élément central du debugging et de l'évaluation des agents : elles enregistrent les appels de modèles, outils, handoffs et guardrails.

6. Remplacer la progression linéaire par un graphe de maîtrise

Le mauvais modèle serait :

Leçon terminée = concept maîtrisé

Un meilleur modèle serait :

Concept découvert
→ appliqué avec assistance
→ appliqué avec variation
→ retrouvé plus tard
→ utilisé dans un autre contexte
→ intégré dans une mission
→ maintenu en production

Chaque compétence devrait posséder plusieurs types de preuves :

```
mastery_evidence:  
  guided_application: true  
  independent_application: true  
  delayed_retrieval: true  
  cross_context_transfer: true  
  debugging_evidence: true  
  production_evidence: true
```

La plateforme peut ensuite générer automatiquement des missions de remédiation.

Un apprenant ayant réussi à créer un agent avec mémoire, mais incapable de diagnostiquer une corruption de cette mémoire, ne doit pas être considéré comme ayant totalement maîtrisé le sujet.

7. Créer un tuteur IA qui ne détruit pas l'apprentissage

Le risque majeur d'une plateforme consacrée à l'IA est que son propre assistant fournisse immédiatement la solution.

Le tuteur devrait fonctionner avec une échelle d'aide stricte :

1. Demander à l'apprenant de prédire le comportement.
2. Lui faire identifier la zone problématique.
3. Montrer la partie pertinente de la trace.
4. Rappeler le concept nécessaire.
5. Donner une stratégie.
6. Donner un pseudo-code.
7. Montrer une solution partielle.
8. Révéler la solution complète en dernier recours.

Le tuteur ne devrait jamais commencer par réécrire le système.

Il devrait également distinguer :

- une erreur de compréhension ;
- une erreur d'implémentation ;
- une mauvaise architecture ;
- une eval incorrecte ;
- une panne externe ;
- un comportement probabiliste normal.

L'objectif n'est pas que l'apprenant termine rapidement. L'objectif est qu'il puisse expliquer **pourquoi le système fonctionne ou échoue**.

8. Construire un curriculum vivant et versionné

Un cours traditionnel peut rester relativement stable plusieurs années. Un cours sur les agents peut devenir obsolète en quelques mois.

Chaque unité devrait donc contenir :

```
curriculum_metadata:
  stable_concept: "Tool routing"
  implementation_track: "OpenAI Agents SDK"
  tested_versions:
    sdk: "x.y.z"
    model: "model-name"
  last_verified_at: "date"
  status: "current | legacy | deprecated | experimental"
  expected_lifetime: "stable | medium | volatile"
```

Le curriculum doit séparer :

Les fondations durables

- state machines ;
- graphes d'exécution ;

- concurrence ;
- systèmes distribués ;
- contrats ;
- idempotence ;
- permissions ;
- observabilité ;
- évaluation ;
- théorie du contrôle ;
- sécurité ;
- économie des systèmes.

Les implémentations temporaires

- SDK particulier ;
- modèle particulier ;
- syntaxe d'un framework ;
- fonctionnalité expérimentale ;
- produit ou fournisseur.

Ainsi, la plateforme n'apprend pas seulement à utiliser LangGraph, Claude Code, Codex ou un Agents SDK. Elle apprend les principes permettant de reconstruire ces systèmes.

9. Rendre les environnements volontairement imparfaits

Les sandboxes ne doivent pas être trop propres.

Les missions avancées doivent introduire :

- rate limits ;
- timeouts ;
- outils indisponibles ;
- réponses partielles ;
- données contradictoires ;
- contexte trop volumineux ;
- credentials expirés ;
- API payante ;
- limites de concurrence ;
- résultat de grader incohérent ;
- utilisateur ambigu ;
- action irréversible nécessitant une approbation.

L'apprenant doit apprendre à construire des systèmes qui échouent proprement.

Les agents efficaces doivent notamment intégrer des critères de succès clairs, des feedback loops et un contrôle humain lorsque nécessaire.

10. Enseigner le coût comme une contrainte fonctionnelle

Un exercice ne devrait pas seulement demander :

Obtiens le bon résultat.

Il devrait demander :

Obtiens le bon résultat avec un taux de réussite supérieur à 90 %, en moins de 30 secondes et pour moins de 0,10 € par exécution.

Les concepts suivants doivent être enseignés dès le départ :

- budget de tokens ;
- cache ;
- sélection du modèle ;
- parallélisation ;
- retries ;
- limites d'itérations ;
- coût des graders ;
- coût des sous-agents ;
- rapport qualité/coût ;
- dégradation contrôlée.

Une architecture brillante mais économiquement impossible doit être considérée comme incorrecte.

11. Être réellement francophone, pas simplement traduit

La plateforme doit fournir un double vocabulaire :

condition d'arrêt – stopping condition
transfert de contrôle – handoff
mise à la terre documentaire – grounding
trace d'exécution – execution trace
évaluation par modèle – model-based grader
mémoire de travail – working memory
état partagé – shared state

L'apprenant doit pouvoir travailler en français tout en comprenant la terminologie anglaise utilisée dans les documentations, repositories et articles techniques.

Les projets doivent également correspondre au marché francophone :

- automatisation de PME ;
- support client francophone ;
- traitement de documents administratifs ;
- agents internes ;
- collectivités ;
- formation ;
- e-commerce ;
- cabinets professionnels ;
- entreprises européennes.

La conformité ne doit pas être un module optionnel ajouté à la fin. En Europe, les règles de gouvernance et plusieurs obligations de l'AI Act sont applicables ou deviennent exécutoires à partir du 2 août 2026. La CNIL souligne également que le passage aux systèmes agentiques augmente les risques liés à la cybersécurité et aux données personnelles.

Chaque mission avancée devrait donc inclure :

- données personnelles ;
- permissions ;
- journalisation ;
- rétention ;
- transparence ;
- validation humaine ;
- actions irréversibles ;
- minimisation des données.

12. Rester indépendant des modèles et des fournisseurs

Une compétence ne devrait pas être validée uniquement parce que l'apprenant sait utiliser un SDK particulier.

Les missions importantes devraient proposer plusieurs implémentations :

```
Track A: fournisseur commercial
Track B: autre fournisseur commercial
Track C: modèle open source ou local
Track D: orchestration manuelle sans framework
```

Certaines missions devraient demander de migrer un même système :

- d'un fournisseur à un autre ;
- d'un gros modèle vers un plus petit ;
- d'une API cloud vers un modèle local ;
- d'un agent unique vers un graphe ;

- d'un framework vers une implémentation maison.

Cela vérifie que l'apprenant maîtrise le système et non la documentation d'une entreprise.

13. Remplacer le certificat par un dossier de preuves

Un certificat seul possède peu de valeur pour de l'Agentic Engineering.

Chaque projet final devrait produire un **evidence pack** :

```
evidence_pack:
  repository: required
  architecture_document: required
  runnable_demo: required
  eval_dataset: required
  eval_results: required
  traces: required
  cost_report: required
  threat_model: required
  failure_analysis: required
  postmortem: required
  model_migration_test: required
  human_review: required
```

Le portfolio doit montrer non seulement que le système fonctionne, mais aussi :

- dans quelles conditions ;
- avec quel taux de réussite ;
- pour quel coût ;
- avec quelles limites ;
- comment il se comporte lorsqu'il échoue ;
- comment il est surveillé ;
- comment il pourrait être amélioré.

14. Positionnement final recommandé

La promesse ne devrait pas être :

Apprenez l'IA étape par étape.

Elle devrait être :

Apprenez à concevoir, évaluer et opérer des systèmes IA fiables.

La différence essentielle peut être résumée ainsi :

freeCodeCamp apprend à produire du code qui passe les tests.

La plateforme doit apprendre à produire des systèmes IA
qui restent fiables lorsque les modèles, les données,
les outils et les conditions changent.

Modèle pédagogique final

Learn the primitive.
Build the loop.
Connect the graph.
Complete the mission.
Operate the system.
Prove its reliability.

C'est cette dernière étape — **prouver la fiabilité** — qui constituerait le principal avantage concurrentiel de la plateforme.